

AARHUS UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

COMPILATION

Hand-in 1

Author

Søren M. DAMSGAARD

Student number

202309814

September 7, 2026



Evaluator

Question 1

Does this function need to recursively explore its arguments, and why (or why not)?

The function `eval` explicitly uses recursion to make it possible for `BinOp` to chain expressions, allowing for longer expressions with multiple operations.

Question 2

Why does this function have the return type `int`? What other return types may be suitable?

Since the function input is an `'expr'` and its type is defined as resulting in a `Int` via either $Int \rightarrow Int$ or its `BinOp` which eventually leads to an `Int`.

Other things that might work, would be types such as `Float` for more precise `Div` operations, or it could also return `None` in case of some error, like division by zero.

Question 3

How does your evaluation handle the case of division by zero? Note that it may be just fine to not special-treat division by zero, but it is important you understand what actually happens at runtime.

No work was put towards treating the division by zero case, since it's compiled by the in-house OCaml compiler it will raise an exception right now, but if the compiler wasn't there to handle it, there will be a funeral.

Pretty Printer

Question 4

Are the functions you have defined recursive, and why (or why not)?

The function is recursive for the same reason as the evaluator function is recursive, it has to traverse down the AST to find the lowest `Int` and make sure to print from the left side of the tree, while it slowly bubbles up throughout the tree.

Question 5

Skipped this one. -Aura #StupidestProgrammerInTheNorth #ReadyToBeWhippedIntoShape

Question 6

What is the OCaml type of `asm_example`?

The type is *elem_list*, but it can be traced to the type *asm* in the *x86.ml* file, where it is defined as lists of various datatypes such as opcodes, operand lists and `ascii`, so it's appropriate to call it a list of elements.

Question 7

How would you rewrite this to use the `Asm` module without the local module open directive?

The local module open directive can be replaced by dot notation instead, as an example for accessing *gtext* in dot notation it could be `X86.Asm.gtext`, this can be used for the whole statement.

```
(X86.Movq, [X86.Asm.( $) 23; X86.Asm( %) X86.Rax])
```

This is ugly, so the local module open directive is preferred. Rather it's more readable.

Question 8

Could we have used `text` instead of `gtext`? Why?

The *gtext* and *text* are helper functions from *x86* but they only differ in one aspect, they defined the block of data/code as either global or local, hence the *g* in *gtext*, this essentially defines if the data/code is public or private, if it's available for other modules to call upon or not, so *gtext* for Social, or *text* for shy

Question 9

What output do you get from `echo $?` when returning the value 2023?

Changing the assembly literal to 2023 makes function output 231, the reasoning is for this is explained in the assignment itself, POSIX only reserve only one byte for the return value, hence the cut-off

Translation

These are the expression made the Evaluator:

$$20 + 26 * 58 = 1528 \quad (1)$$

$$5 * 1/10 = 0 \quad (2)$$

$$31 - 870 = -839 \quad (3)$$

$$6/10 + 49 + 10 * 70 * 77 - 12/9 + 5 = 53940 \quad (4)$$

$$34/72 - 17/46 = 0 \quad (5)$$

To run the OCaml evaluator and print the generated assembly strings, and run the command `dune exec main.exe`.

Here it's very apparent that the syntax is missing parantheses, although the correct semantic of them remain, hence the output is still correct.

The results were cross-referenced by manually creating the assembly files and running them through the C wrapper, checking the output manually like with earlier example given in the assignment.

How to run the code

You will have received a zip which contains:

1. Minimum dune files
2. Assembly files for each expression in the assignment
3. C wrapper to Assembly, main.c
4. The x86.ml file and the main.ml file, which contains all the code

To run the main file, and generate the assembly code and the various expression follow the steps: (Beware: A lot of the expressions and assembly code is printed in the terminal):

1. Unzip the project inside the development container given in the course (docker).
2. Move inside the project folder named `handin_1`.
3. Run the following command in the terminal `dune exec ./main.exe` to run the main file.
4. See your terminal be flooded.

To test each assembly file, you have to change a small line in the C wrapper, main.c, and then run `clang main.c task3.exp1.s` where you can change the number on the assembly file correlating to the expression you want tested. After each compile, check the output file 'a.out' by using the command `./a.out` in the terminal.